LAB ANALYSIS: Cleartext Credential Capture on DVWA

Wireshark Week, Task 6 | Analyst: Kunal | Date: 2026-11-09 | Environment: Kali VM, DVWA on localhost (127.0.0.1:42001)

1. OBJECTIVE

Capture an HTTP login to a locally hosted DVWA and show what an attacker on the network could read.

2. METHOD

- Interface **lo** (loopback), no capture filter. Login captured at **2026-11-09 08:22:42 UTC** (from the server's Date header).
- DVWA (Kali package) is served by nginx on port **42001**, which Wireshark did not decode as HTTP at first. Fixed with *Analyze* > *Decode As: TCP port 42001* > *HTTP*.
- Display filter: `http.request.method == "POST"` (fallbacks: `tcp.port == 42001`, `frame contains "password"`).
- Read the fields with *HTML Form URL Encoded* and *Follow TCP Stream* (tcp.stream eq 0).

3. FINDINGS (EVIDENCE)

Item	Value captured	Where found
Request	POST /login.php HTTP/1.1 to 127.0.0.1:42001 (application/x-www-form-urlencoded, 88 bytes)	Request line and headers
Username	admin	HTML Form URL Encoded
Password	pa****rd (masked here; fully readable in the capture)	HTML Form URL Encoded
CSRF token	user_token=182eb1fc...469e (shortened). Defends against forged requests, not against sniffing.	HTML Form URL Encoded
Session cookie, before login	PHPSESSID=fced00fb...c774	Cookie header of the POST
Session cookie, after login	PHPSESSID=82e2e435...abccd. Flags: HttpOnly, SameSite=Strict. **No Secure flag.**	Set-Cookie header of the 302
Login result: success	302 Found, Location: index.php, then GET /index.php sent with the new cookie	Response status and Location
Login result: wrong password	302 Found, Location: login.php (back to the login form)	Response in the second capture

```
POST /login.php HTTP/1.1
Host: 127.0.0.1:42001
Content-Type: application/x-www-form-urlencoded
Cookie: security=impossible; PHPSESSID=fced00fb...c774

username=admin&password=pa****rd&Login=Login&user_token=182eb1fc...469e

HTTP/1.1 302 Found
Set-Cookie: PHPSESSID=82e2e435...abccd; Max-Age=86400; path=/;
            domain=127.0.0.1; HttpOnly; SameSite=Strict
Location: index.php
```

Figure 1. Text transcribed from the Follow TCP Stream window; secrets masked or shortened.

Observations

- The session ID **changed after login** (fced00fb... to 82e2e435...). Issuing a fresh session on login limits session fixation.
- **HttpOnly** and **SameSite=Strict** are set, but **Secure** is not, so the cookie crosses the network in cleartext and a sniffer can replay it.
- The cookie `security=impossible` shows DVWA's hardest level was active, yet the credentials still travelled in cleartext. Hardened code does not replace TLS.

4. IMPACT

Anyone who can observe this traffic (shared Wi-Fi, a compromised router, ARP spoofing) can read the username and password and replay the session cookie to log in as this user. In production the risk would be **High**, because passwords are often reused on other services. Here the credentials are DVWA's default lab login, so there is no real exposure.

5. RECOMMENDATIONS (BY PRIORITY)

1. Serve the whole site over **HTTPS**, redirect HTTP to HTTPS, and enable **HSTS**. This fixes the actual finding.
2. Add the **Secure** flag to the session cookie (HttpOnly and SameSite=Strict are already set).
3. Offer **MFA**, so a sniffed password alone is not enough.
4. Store passwords with **bcrypt or Argon2**. This is defence in depth for a database leak; it does not protect the wire.
5. Alert on logins submitted over plain HTTP.

6. LIMITS

- Loopback lab with default credentials. On a real network the attacker also needs a viewing position (same Wi-Fi, a compromised router, ARP spoofing).
- One successful and one failed login were examined; the failed-login result comes from the second capture. Wireshark reported "1 bytes missing in capture file" at the start of the stream, which did not affect the fields above.